



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Trabalho Prático - Grupo 31

Afonso Martins
a106931

Gonçalo Castro
a107337

Francisco Lage
a106813

Ricardo Neves
a106850

Fevereiro, 2026

CG

Resumo

Este documento descreve o desenvolvimento da 1ª fase do projeto prático da Unidade Curricular de Computação Gráfica, lecionada no 2.º semestre do 3.º ano da Licenciatura em Engenharia Informática.

O objetivo principal desta fase foi a implementação de dois módulos fundamentais: o **generator** e o **engine**. Para além disso, foram desenvolvidas primitivas geométricas básicas obrigatórias (plano, caixa, esfera e cone). Como extensão, foram ainda implementadas primitivas adicionais e funcionalidades complementares.

Índice

1. Arquitetura do Projeto	1
2. Programa <i>Generator</i>	1
2.1. Formato do ficheiro .3d	1
2.2. Plano	2
2.3. Esfera	2
2.4. Cone	3
2.5. Caixa	4
2.6. Cilindro	5
2.7. Torus	5
2.8. Icosfera	6
3. Programa <i>Engine</i>	6
3.1. Renderização de Modelos	7
3.1.1. Parsing de .obj	7
3.2. Câmara	7
3.3. Configurações da UI	8
4. Testes	8
5. Conclusão	10

Lista de Figuras

Figura 1	Geração de um plano	2
Figura 2	Geração de uma esfera	3
Figura 3	Geração de um cone	4
Figura 4	Geração de uma caixa	4
Figura 5	Geração de um cilindro	5
Figura 6	Geração de um torus	6
Figura 7	Geração de uma icosfera	6
Figura 8	Interface Gráfica de configurações apresentada pelo Engine	8
Figura 9	Execução do test_1_1	9
Figura 10	Execução do test_1_2	9
Figura 11	Execução do test_1_3	9
Figura 12	Execução do test_1_4	9
Figura 13	Execução do test_1_5	9

1. Arquitetura do Projeto

O sistema encontra-se dividido em duas aplicações independentes:

- **Generator:** responsável pela criação de ficheiros .3d contendo a definição geométrica das primitivas.
- **Engine:** responsável pela leitura de ficheiros XML de configuração e pela renderização dos modelos previamente gerados.

O fluxo do sistema inicia-se com a geração dos modelos geométricos, que são posteriormente referenciados num ficheiro de cena em formato XML e interpretados pelo motor gráfico.

2. Programa *Generator*

O módulo **generator** produz ficheiros .3d contendo a descrição dos vértices necessários para representar cada primitiva.

A invocação do programa é feita através da linha de comandos, indicando:

- Tipo de primitiva
- Parâmetros geométricos
- Nome do ficheiro de saída

```
> ./build/generator/generator
Usage: generator <command> <args> <output file>
Commands:
    generator plane <length> <divisions> <output file>
    generator sphere <radius> <slices> <stacks> <output file>
    generator cone <radius> <height> <slices> <stacks> <output file>
    generator box <length> <divisions> <output file>
    generator cylinder <radius> <height> <slices> <stacks> <output file>
    generator torus <radius> <tubeRadius> <slices> <stacks> <output file>
    generator icosphere <radius> <subdivisions> <output file>
```

2.1. Formato do ficheiro .3d

No início do ficheiro denota-se o valor total de vértices que se vão seguir para otimizações no posterior processamento do mesmo. De seguida, cada linha representa um vértice no formato:

x y z

Por exemplo:

```
1
0 0.2 0.9
0 0 1
0.309017 0 0.951057
```

Cada conjunto de três vértices consecutivos define um triângulo. O exemplo acima descreve um só triângulo de coordenadas específicas.

Esta estrutura modular permite futura expansão para incluir outros atributos que sejam necessários.

2.2. Plano

O plano é gerado no plano XZ e encontra-se centrado na origem. A sua construção depende de dois parâmetros: o comprimento total do lado (length) e o número de subdivisões por eixo (divisions). Estes valores determinam a resolução da malha triangular que compõe a superfície.

O tamanho de cada subdivisão é calculado por:

$$\text{divisionSize} = \frac{\text{length}}{\text{divisions}}$$

Para garantir que o plano fica centrado na origem, considera-se metade do comprimento:

$$\text{halfLength} = \frac{\text{length}}{2}$$

A construção é realizada iterando sobre dois índices, i e j , correspondentes às direções dos eixos x e z , respetivamente.

Para cada célula (i, j) são definidos quatro vértices:

$$P_{\{tl\}} = (-\text{halfLength} + i \cdot \text{divisionSize}, 0, -\text{halfLength} + j \cdot \text{divisionSize})$$

$$P_{\{tr\}} = (-\text{halfLength} + (i + 1) \cdot \text{divisionSize}, 0, -\text{halfLength} + j \cdot \text{divisionSize})$$

$$P_{\{bl\}} = (-\text{halfLength} + i \cdot \text{divisionSize}, 0, -\text{halfLength} + (j + 1) \cdot \text{divisionSize})$$

$$P_{\{br\}} = (-\text{halfLength} + (i + 1) \cdot \text{divisionSize}, 0, -\text{halfLength} + (j + 1) \cdot \text{divisionSize})$$

Cada célula é composta por dois triângulos:

$$(P_{\{tl\}}, P_{\{bl\}}, P_{\{br\}})$$

$$(P_{\{tl\}}, P_{\{br\}}, P_{\{tr\}})$$

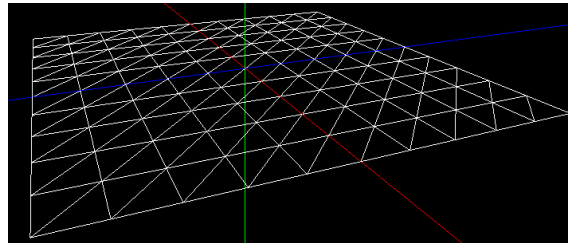


Figura 1: Geração de um plano

2.3. Esfera

A esfera é centrada na origem e é definida pelo seu raio (radius), pelo número de divisões longitudinais (slices) e pelo número de divisões latitudinais (stacks). Estes parâmetros controlam o nível de detalhe da malha esférica.

São utilizadas coordenadas esféricas convertidas para coordenadas cartesianas através das seguintes expressões:

$$x = r \cos(\beta) \sin(\alpha)$$

$$y = r \sin(\beta)$$

$$z = r \cos(\beta) \cos(\alpha)$$

Os ângulos são definidos por:

$$\alpha = \text{slice} \cdot \frac{2\pi}{\text{slices}}$$

$$\beta = \text{stack} \cdot \frac{\pi}{\text{stacks}} - \frac{\pi}{2}$$

Cada célula paramétrica da esfera gera dois triângulos, com exceção das regiões próximas dos polos, onde apenas um triângulo é necessário.

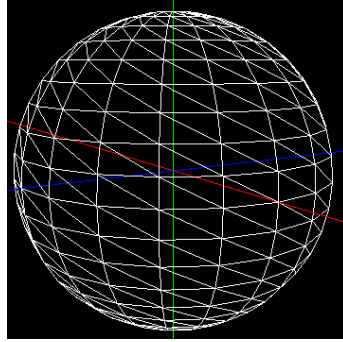


Figura 2: Geração de uma esfera

2.4. Cone

O cone é construído com base no raio da base (radius), na altura (height), no número de divisões angulares (slices) e no número de subdivisões verticais (stacks). Estes parâmetros determinam a discretização da superfície lateral e da base.

A discretização angular é dada por:

$$\text{sliceSize} = \frac{2\pi}{\text{slices}}$$

A subdivisão ao longo da altura é:

$$\text{stackSize} = \frac{\text{height}}{\text{stacks}}$$

O raio varia linearmente ao longo da altura:

$$r_{\{\text{atual}\}} = \text{radius} - \text{stack} \cdot \frac{\text{radius}}{\text{stacks}}$$

$$r_{\{\text{seguinte}\}} = \text{radius} - (\text{stack} + 1) \cdot \frac{\text{radius}}{\text{stacks}}$$

Cada subdivisão lateral gera dois triângulos, exceto no nível superior, onde apenas um é necessário para fechar a geometria.

A base é construída ligando o ponto central $(0, 0, 0)$ aos pontos consecutivos da circunferência.

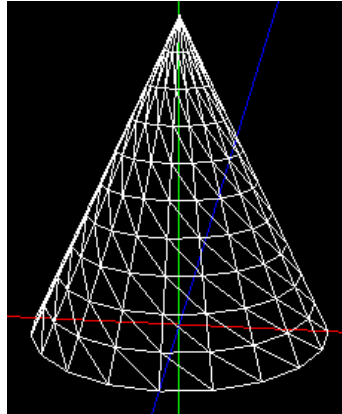


Figura 3: Geração de um cone

2.5. Caixa

A caixa é centrada na origem e definida pelo comprimento da aresta (length) e pelo número de subdivisões por face (divisions). Em vez de gerar cada face individualmente, reutiliza-se a malha do plano, aplicando transformações geométricas adequadas.

Primeiramente é gerado um plano subdividido. Posteriormente, esse plano é transformado seis vezes através de matrizes de translação e rotação, permitindo posicionar corretamente cada face da caixa:

- Superior
- Inferior
- Esquerda
- Direita
- Frente
- Trás

Cada face mantém a mesma estrutura de subdivisão do plano original.

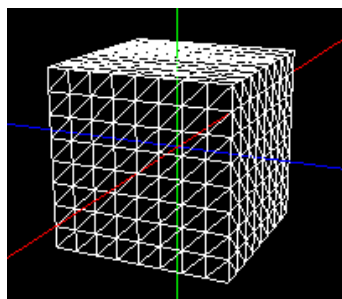


Figura 4: Geração de uma caixa

2.6. Cilindro

O cilindro é centrado na origem e é definido pelo raio (radius), pela altura (height), pelo número de divisões angulares (slices) e pelo número de subdivisões verticais (stacks).

A discretização angular é dada por:

$$\theta = \text{slice} \cdot \frac{2\pi}{\text{slices}}$$

A altura é distribuída uniformemente no intervalo:

$$\left[-\frac{\text{height}}{2}, \frac{\text{height}}{2} \right]$$

A superfície lateral é formada por células retangulares que originam dois triângulos:

$$\left(\text{bottom}_{\{\text{left}\}}, \text{bottom}_{\{\text{right}\}}, \top_{\{\text{left}\}} \right)$$

$$\left(\top_{\{\text{left}\}}, \text{bottom}_{\{\text{right}\}}, \text{top}_{\{\text{right}\}} \right)$$

As bases são construídas ligando o centro da circunferência aos pontos adjacentes do perímetro.

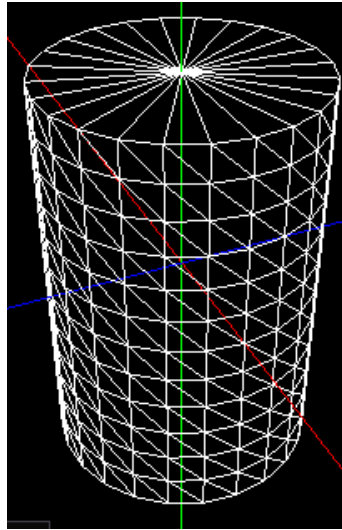


Figura 5: Geração de um cilindro

2.7. Torus

O torus é caracterizado pelo raio principal (radius), pelo raio do tubo (tubeRadius), e pelos parâmetros de discretização slices e stacks.

São utilizados dois ângulos paramétricos:

$$\theta = \frac{2\pi \cdot \text{stack}}{\text{stacks}}$$

$$\varphi = \frac{2\pi \cdot \text{slice}}{\text{slices}}$$

As equações paramétricas do torus são:

$$x = (\text{radius} + \text{tubeRadius} \cos \varphi) \cos \theta$$

$$y = \text{tubeRadius} \sin \varphi$$

$$z = (\text{radius} + \text{tubeRadius} \cos \varphi) \sin \theta$$

Cada célula da malha paramétrica gera dois triângulos.

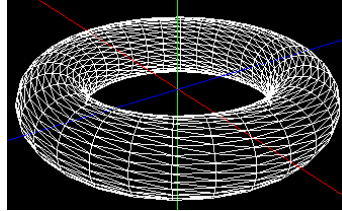


Figura 6: Geração de um torus

2.8. Icosfera

A icosfera é gerada a partir de um icosaedro regular inicial, sendo definida pelo raio final (radius) e pelo número de subdivisões (subdivisions).

Inicialmente são definidos os 12 vértices do icosaedro e as 20 faces triangulares. Para cada subdivisão adicional, cada triângulo é dividido em quatro novos triângulos através do cálculo dos pontos médios das suas arestas, seguido de normalização dos novos vértices.

Após todas as subdivisões, cada vértice é escalado:

$$\text{vertex}_{\{\text{final}\}} = \text{vertex}_{\{\text{normalizado}\}} \cdot \text{radius}$$

O número total de triângulos cresce segundo:

$$T_n = 20 \cdot 4^{\{(n-1)\}}$$

O que implica crescimento exponencial do custo computacional.

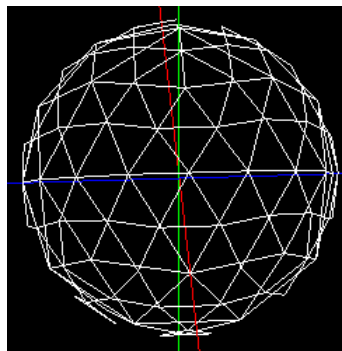


Figura 7: Geração de uma icosfera

3. Programa *Engine*

O *engine* é a aplicação responsável pela renderização de cenas compostas por modelos 3D (.3d e .obj).

As cenas são definidas em ficheiros .xml que contêm a seguinte informação:

- **Janela:** largura e altura
- **Câmara:** posição, ponto de foco, vetor *up*, FOV, *near* e *far*
- **Grupos:** hierarquia de grupos, cada um com transformações e modelos

Estes ficheiros são interpretados com recurso à biblioteca **tinyxml2**. A estrutura da cena foi implementada de forma hierárquica através de grupos, o que permite suportar transformações encadeadas e facilita a extensão nas fases seguintes.

3.1. Renderização de Modelos

Na fase de inicialização, o *engine* lê todos os modelos referenciados na cena e carrega os seus vértices para memória. Cada modelo é guardado como uma lista plana de vértices, onde cada grupo de três consecutivos define um triângulo.

A cada *frame*, o programa percorre a hierarquia de grupos recursivamente. Para cada grupo, aplica as suas transformações usando a pilha de matrizes do OpenGL — `glPushMatrix` antes e `glPopMatrix` depois — garantindo que cada grupo filho herda as transformações do pai sem afectar os restantes. Os triângulos são depois enviados à GPU em modo imediato com `glBegin(GL_TRIANGLES)`.

Esta abordagem repete vértices em memória, não sendo ideal, mas será optimizada com VBOs numa fase seguinte.

3.1.1. Parsing de .obj

Para além do formato nativo `.3d`, o *engine* suporta também ficheiros Wavefront `.obj`, detectados automaticamente pela extensão. O *parser* lê os vértices e resolve as faces para uma lista plana de triângulos, suportando o formato composto `v/vt/vn` mas usando apenas o índice de posição. Funciona para modelos com faces exclusivamente triangulares.

3.2. Câmara

A câmara é configurada inicialmente a partir do ficheiro `.xml` da cena e opera em modo **orbital**: o utilizador orbita em torno de um ponto fixo de interesse. O estado é guardado em coordenadas cartesianas, mas o processamento de *input* trabalha em coordenadas esféricas (ρ, α, β) , onde ρ é a distância ao ponto de foco, α o ângulo horizontal e β o ângulo vertical.

A cada evento de *input*, a posição é convertida para esféricas, os valores modificados e a nova posição recalculada:

$$\mathbf{P}' = \mathbf{L} + \begin{pmatrix} \rho \cos \beta \sin \alpha \\ \rho \sin \beta \\ \rho \cos \beta \cos \alpha \end{pmatrix}$$

O botão esquerdo do rato controla α e β , e a roda controla ρ , com sensibilidades de **0.002 rad/px** e **0.2 unidades/tick**. São aplicadas duas restrições: β é limitado a $(-\pi/2 + 0.001, +\pi/2 - 0.001)$ para evitar *gimbal flip*, e $\rho \geq 1.0$ para a câmara não entrar dentro dos objetos.

No início de cada *frame*, a câmara é configurada com `gluPerspective` e `gluLookAt`, com a razão de aspeto recalculada a partir das dimensões atuais do *framebuffer*.

3.3. Configurações da UI

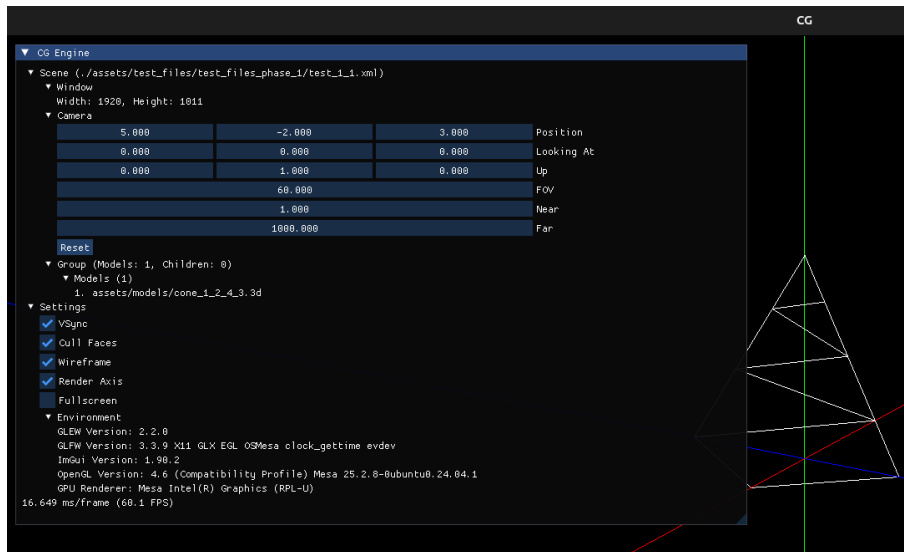


Figura 8: Interface Gráfica de configurações apresentada pelo Engine

A interface gráfica do *engine* apresenta, em tempo real, as principais configurações da cena atualmente carregada, bem como informações de desempenho e ambiente de execução.

Na secção da cena é possível visualizar o ficheiro XML ativo, as dimensões da janela de renderização (largura e altura) e os parâmetros da câmara, incluindo posição, ponto de foco (**lookAt**), vetor **up**, campo de visão (FOV) e planos de corte **near** e **far**. É ainda disponibilizada a opção de reposição (**reset**) da câmara para os valores iniciais definidos no ficheiro de configuração.

A interface permite também consultar os modelos carregados na cena e ativar ou desativar diferentes opções de renderização, tais como VSync, **culling** de faces, modo **wireframe**, visualização dos eixos e modo **fullscreen**.

Por fim, são apresentadas informações técnicas do ambiente gráfico (GLFW, OpenGL, ImGui e GPU utilizada), bem como métricas de desempenho, nomeadamente o tempo médio por frame (ms/frame) e o valor de FPS (**frames per second**), permitindo avaliar a eficiência da implementação.

4. Testes

Esta secção serve simplesmente para demonstrar que o nosso duo **Generator+Engine** foram implementados com sucesso para darem resposta aos resultados pretendidos para o conjunto de *test_files* disponibilizados pela equipa docente para esta 1ª Fase do projeto

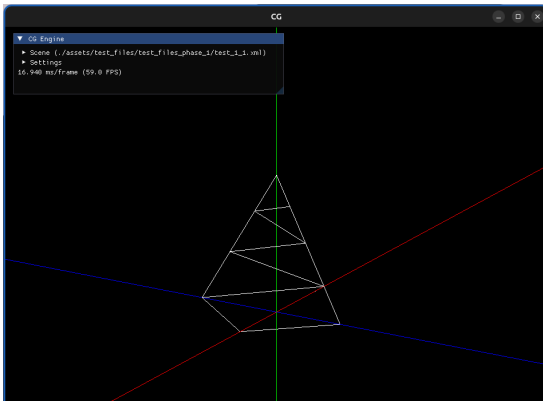


Figura 9: Execução do test_1_1

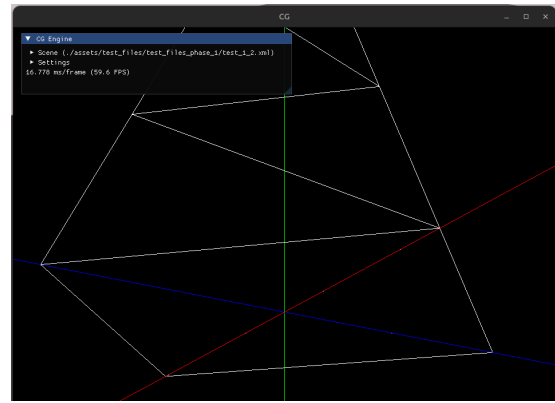


Figura 10: Execução do test_1_2

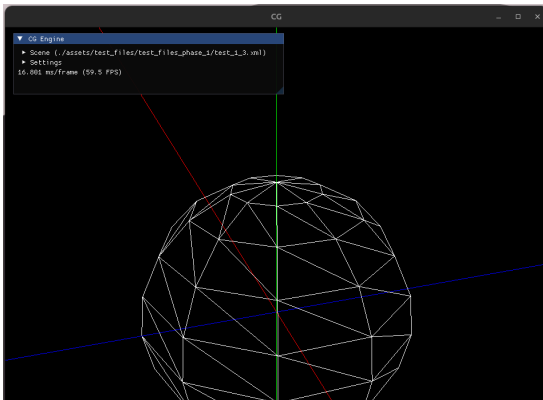


Figura 11: Execução do test_1_3

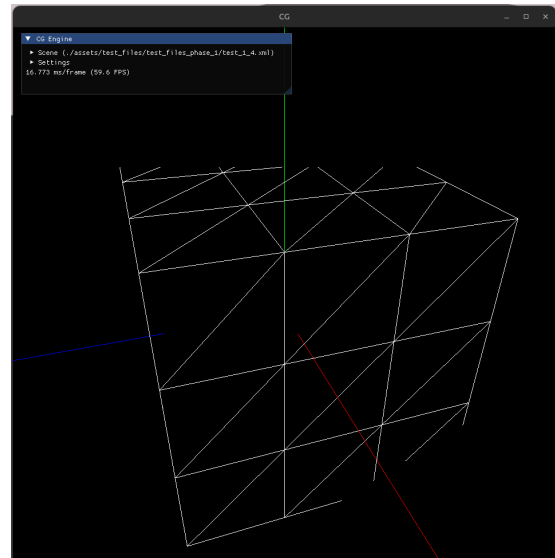


Figura 12: Execução do test_1_4

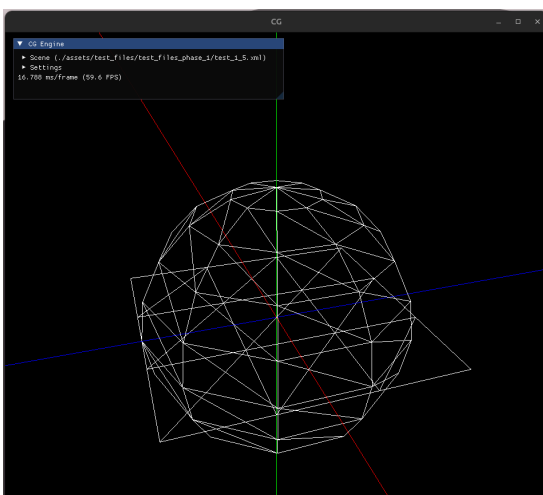


Figura 13: Execução do test_1_5

5. Conclusão

A primeira fase do projeto permitiu consolidar as bases estruturais do motor gráfico, através da implementação das primitivas geométricas obrigatórias e da extensão funcional com novas formas, nomeadamente o cilindro, o torus e a icosfera.

Foi igualmente estabelecida uma arquitetura modular, separando claramente o processo de geração geométrica da fase de renderização, o que promove organização, reutilização de código e facilidade de evolução futura.

O *engine* demonstra já a capacidade de interpretar ficheiros de configuração em XML, carregar modelos previamente gerados (em vários formatos) e renderizar cenas organizadas hierarquicamente. Desta forma, encontra-se criada uma base técnica sólida e escalável para a implementação das funcionalidades previstas nas fases seguintes do projeto.